

University of Aveiro

DETI

Engenharia e Gestão de
Serviços

Mestrado Computadores e
Telemática



universidade de aveiro

Marta Oliveira (97613)

Rafael Amorim (98197)

Tiago Alves (104110)

Luís Couto (89078)

April 20, 2024

Índice

1	Aplicação TouristGuide	3
2	Serviços	4
2.1	Calendarização	4
2.1.1	Introdução	4
2.1.2	Tecnologias	4
2.1.3	Esquema e Implementação	4
2.1.4	Esquema e Implementação da Base de Dados	6
2.1.5	Resultados	7
2.2	Serviço Pontos de Interesse	8
2.2.1	Introdução	8
2.2.2	Tecnologias	8
2.2.3	'Schema' e implementação	8
2.2.4	Resultados	11
2.3	Serviço Metadados	14
2.3.1	Introdução	14
2.3.2	Tecnologias	14
2.3.3	Esquema e Implementação	15
2.3.4	Resultados	17
3	Kubernetes	21
4	Demo	22
4.0.1	Video Demo	22
4.0.2	Implementação	22
4.1	Endpoint Login	23
4.2	Endpoints Pontos de Interesse	24
4.3	Endpoints Eventos	28
5	Autoavaliação	32

List of Figures

1	Arquitetura Geral Projeto	3
2	Documentação do serviço calendário	5
3	Login	7
4	Adicionar evento ao calendário resultante do login anterior	7
5	Eventos do calendário	7
6	Queries Schema	9
7	Mutações Schema	10
8	Chave API gerada	17
9	Chave API armazenada	17
10	Dados de eventos	18
11	Estatísticas de favoritos	18

12	Log de eventos	19
13	Ficheiros no sistema de armazenamento	19
14	Obtenção de um ficheiro do sistema de armazenamento	19
15	Documentação do serviço	20
16	Arquitetura de deployment	21
17	Pods em execução	21
18	Login existent	23
19	Nova conta	23
20	Login	23
21	Todos os pontos de interesse e categoria landmarks seleccionada	24
22	Filters	25
23	Popup vazio	26
24	Popup preenchido	27
25	Chamada da lista de eventos e adição ao calendário	28
26	Chamada da lista de eventos e adição ao calendário	29
27	Eventos por dia	30
28	Evento do Benfica saiu dos favoritos	31
29	Sem eventos no dia 17	31
30	Resultado dos eventos eliminados	31

1 Aplicação TouristGuide

A ideia do projeto consiste na criação de uma aplicação de gestão de eventos que permita a procura e o planeamento dos mesmos numa dada cidade.

Neste sistema é possível:

- Pesquisar pontos de interesse e eventos associados a eles por cidade/região
- Ver informação acerca dos pontos de interesse e/ou eventos
- Serviço de calendarização
 - Se existir alteração dos eventos, estas são refletidas no calendário (sincronismo)

Desta forma, existe uma plataforma única onde a informação está centralizada, com atualizações em tempo real e com sincronização com calendários pessoais para um planeamento eficiente.

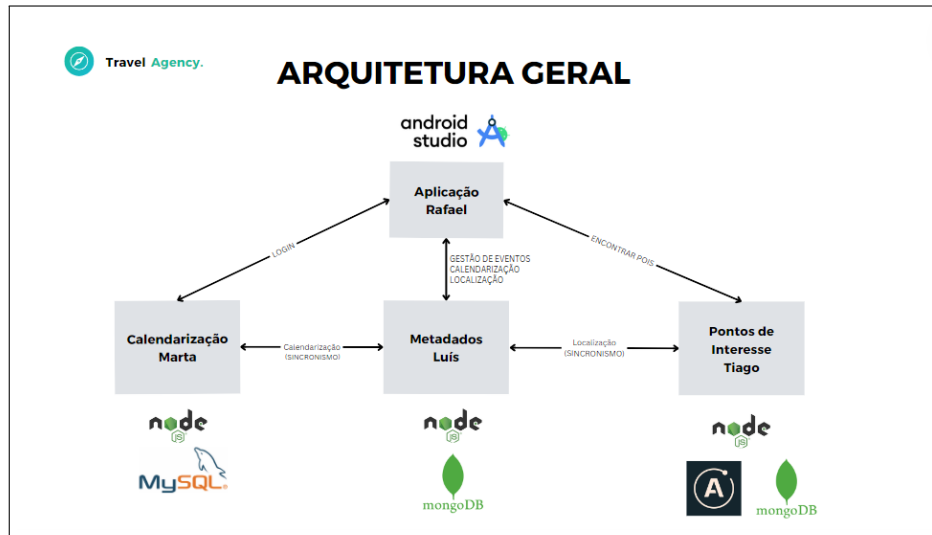


Figure 1: Arquitetura Geral Projeto

Todo o código desenvolvido encontra-se disponível em: <https://github.com/orgs/EGS-Tourist-Guide/repositories>

Apesar de haver comunicação entre serviços para este projeto, cada um consegue funcionar de forma independente (modular).

2 Serviços

2.1 Calendarização

2.1.1 Introdução

O serviço de calendarização foi desenvolvido para gerir calendários e eventos de utilizadores.

2.1.2 Tecnologias

No desenvolvimento deste serviço, as tecnologias utilizadas foram Node.js e MySQL que, em conjunto, proporcionam uma aplicação mais robusta, escalável e eficiente.

Node.js é conhecido pela capacidade de gerir um grande número de conexões simultâneas com uma elevada taxa de transferência. Isto é vantajoso para este tipo de serviço, onde vários utilizadores podem estar a aceder e atualizar/adicionar eventos simultaneamente.

Já o MySQL é conhecido pela facilidade de uso, compatibilidade e desempenho. Estas características são cruciais para este serviço que precisa de garantir a integridade e disponibilidade dos dados dos utilizadores a qualquer momento.

Além disso, este serviço precisa de gerir dados relacionados, como utilizadores, calendários e eventos. O MySQL, sendo uma base de dados relacional, permite definir e manter relações entre estas tabelas de forma eficaz.

2.1.3 Esquema e Implementação

O serviço de Calendarização integra os seguintes endpoints:

- Gerar/Recuperar chaves de serviço garantindo um acesso seguro aos endpoints.
- Associar/Criar ID de um utilizador a um ID de calendário (login)
- Adicionar ou remover eventos do calendário do utilizador
- Procurar eventos por filtro (Data, localização, id do evento)
- Atualizar evento no calendário do utilizador
- Remover eventos do calendário do utilizador

A documentação do serviço encontra-se disponível em: <http://grupo4-egs-deti.ua.pt/api-docs/>.

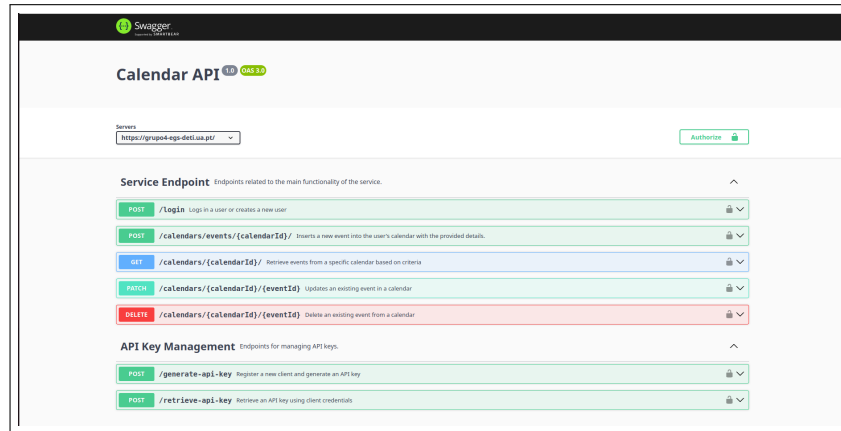


Figure 2: Documentação do serviço calendário

Para efeitos de teste, pode ser usada a chave "7d4d3351-64b9-4d0a-918c-419828a941d3" para autenticação.

Os endpoints do serviço são descritos abaixo:

Endpoints de Gestão de chaves API:

POST /generate-api-key: Regista um novo cliente através da indicação de um username e password. É gerada uma chave API única para autenticação.

POST /retrieve-api-key: Recupera uma chave API existente para um cliente que se já registou anteriormente.

Endpoints de Gestão do Utilizador:

POST /login: Gere o login ou registo do utilizador. Verifica se um utilizador existe, valida as credenciais e cria um calendário se for a primeira vez que o utilizador faz login para lhe ficar associado. Ou seja, é criado um identificador para o utilizador e um identificador para o calendário.

Endpoints de Gestão de Eventos no Calendário:

GET /calendars/:calendarId/: Recupera eventos do calendário especificado, com parâmetros opcionais de consulta(filtrar por data, localização e ID de evento).

POST /calendars/events/:calendarId/: Insere um novo evento no calendário do utilizador.

PATCH /calendars/:calendarId/:eventId: Atualiza um evento existente no calendário do utilizador.

DELETE /calendars/:calendarId/:eventId: Elimina um evento existente do calendário do utilizador.

2.1.4 Esquema e Implementação da Base de Dados

A seguir, vão ser descritas as tabelas principais que foram criadas para a base de dados deste serviço:

apikey: Esta tabela armazena as chaves de API para autenticação dos clientes que utilizam este serviço. Cada registo contém um nome de utilizador, senha(hashado) e a chave API gerada.

- **id:** Identificador único da chave.
- **username:** Nome do utilizador.
- **password:** Password do utilizador(hashado).
- **apikey:** Chave gerada.

users: Esta tabela contém informações sobre os utilizadores do serviço. Cada utilizador é identificado pelo seu ID e tem um endereço de email único.

- **id:** Identificador único do utilizador.
- **email:** Endereço de email do utilizador.
- **name:** Nome do utilizador.

calendars: Esta tabela armazena os calendários dos utilizadores. Cada calendário está associado a um utilizador através do seu ID.

- **id:** Identificador único do calendário.
- **userId:** Identificador do utilizador associado ao calendário.
- **name:** Nome do calendário(opcional).

events: Esta tabela armazena os eventos associados aos calendários dos utilizadores. Cada evento possui um identificador único.

- **id:** Identificador único do evento.
- **calendarId:** Identificador do calendário ao qual o evento pertence.
- **summary:** Resumo do evento.
- **location:** Localização do evento.
- **description:** Descrição do evento.
- **startDateTime:** Data e hora do início do evento.
- **endDateTime:** Data e hora do fim do evento.
- **timeZone:** Fuso horário do evento.
- **obs:** Observações adicionais (campo opcional).

2.1.5 Resultados

É possível verificar o funcionamento deste serviço na página da documentação. Aqui neste capítulo estão demonstrados alguns dos endpoints para efeitos de demonstração.



```
curl -X 'POST' \
  'https://grup04-egs-deti.ua.pt/login' \
  -H 'accept: application/json' \
  -H 'x-api-key: 7d4d3351-64b9-4d8a-918c-419828a941d3' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "d99cecamp@ua.com",
    "password": "test@123"
  }'
```

Request URL

https://grup04-egs-deti.ua.pt/login

Server response

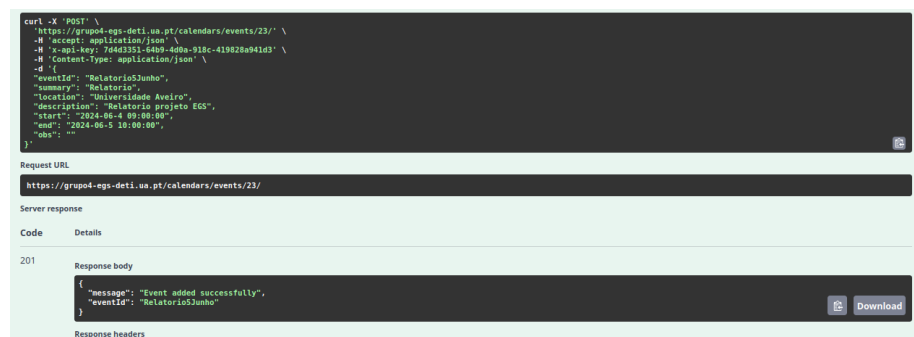
Code Details

201

Response body

```
{
  "message": "Login successfully",
  "userId": 23,
  "calendarId": 23
}
```

Figure 3: Login



```
curl -X 'POST' \
  'https://grup04-egs-deti.ua.pt/calendars/events/23/' \
  -H 'accept: application/json' \
  -H 'x-api-key: 7d4d3351-64b9-4d8a-918c-419828a941d3' \
  -H 'Content-Type: application/json' \
  -d '{
    "eventId": "Relatorio3Junho",
    "summary": "Relatorio",
    "location": "Universidade Aveiro",
    "description": "Relatorio projeto EGS",
    "start": "2024-06-4 09:00:00",
    "end": "2024-06-5 18:00:00",
    "obs": ""
  }'
```

Request URL

https://grup04-egs-deti.ua.pt/calendars/events/23/

Server response

Code Details

201

Response body

```
{
  "message": "Event added successfully",
  "eventId": "Relatorio3Junho"
}
```

Response headers

Figure 4: Adicionar evento ao calendário resultante do login anterior



```
curl -X 'GET' \
  'https://grup04-egs-deti.ua.pt/calendars/23/' \
  -H 'accept: application/json' \
  -H 'x-api-key: 7d4d3351-64b9-4d8a-918c-419828a941d3'
```

Request URL

https://grup04-egs-deti.ua.pt/calendars/23/

Server response

Code Details

200

Response body

```
{
  "eventId": "Relatorio3Junho",
  "summary": "Relatorio",
  "location": "Universidade Aveiro",
  "description": "Relatorio projeto EGS",
  "startDateTime": "2024-06-04 09:00:00",
  "endDateTime": "2024-06-05 18:00:00"
}
```

Figure 5: Eventos do calendário

2.2 Serviço Pontos de Interesse

2.2.1 Introdução

A ideia fundamental deste serviço é proporcionar aos utilizadores, essencialmente turistas, uma procura específica de pontos de interesse (POIs) para os ajudar enquanto viajam. Os turistas devem poder procurar por pontos de interesse mais próximos de uma certa localização geográfica (seleccionando um raio de procura), nome do local e/ou categoria.

2.2.2 Tecnologias

Foi utilizado GraphQL para a realização da API deste serviço. A escolha de GraphQL em vez de REST foi devido ao facto do serviço ter um foco principal, a procura de pontos de interesse. Com a aplicação de diversos filtros, os utilizadores têm a possibilidade de obter a informação exata que pretendem sobre POIs utilizando um único endpoint.

Como resultado desta decisão foram escolhidas NodeJS e Apollo Server, duas tecnologias que simplificam o processo de construção de APIs em GraphQL. Estas são ambas bem documentadas e relativamente fáceis de usar. Apollo contém ainda um IDE focado em operações GraphQL que foi muito útil para efeitos de depuração durante o projeto.

Quanto à base de dados foi escolhido MongoDB, para criação de uma base de dados não relacional. Esta tecnologia consegue ser facilmente integrada com o Apollo Server e NodeJS, permitindo uma obtenção e manutenção eficiente dos dados. Além disso, alguma informação dos pontos de interesse não é obrigatória e, por essa razão, pode estar ausente. Como o MongoDB não requer espaço para guardar valores inexistentes (como null), há uma clara vantagem no uso deste em termos de eficiência de armazenamento dos dados. Por último MongoDB permite a utilização de queries geoespaciais, com o uso de índices geoespaciais esféricos 2D.

2.2.3 'Schema' e implementação

A documentação de uma API GraphQL pode ser dada pelo seu 'schema', pelo que abaixo é apresentado um excerto com as partes mais importantes do mesmo.

```
export const typeDefs = `#graphql
  type PointOfInterest {
    _id: ID!
    name: String!
    location: Point!
    locationName: String!
    street: String
    postcode: String
    description: String
    category: String
    thumbnail: String
  }

  input PoiSearchInput {
    _id: ID
    location: PointInput
    radius: Float
    locationName: String
    category: String
  }

  type Point {
    type: String!
    coordinates: [Float]!
  }

  type Query {
    searchPointsOfInterest(searchInput: PoiSearchInput,
      apiKey: String!): [PointOfInterest!]!

    recoverApiKey(clientName: String!, password: String!): String!
  }
`
```

Figure 6: Queries Schema

```

input CreatePointOfInterestInput {
  name: String!
  location: PointInput!
  locationName: String!
  street: String
  postcode: String
  description: String
  category: String
  thumbnail: String
}

input UpdatePointOfInterestInput {
  name: String
  location: PointInput
  locationName: String
  street: String
  postcode: String
  description: String
  category: String
  thumbnail: String
}

type Mutation {
  createPointOfInterest(input: CreatePointOfInterestInput!,
    apiKey: String!): PointOfInterestWithMessage!
  updatePointOfInterest(_id: ID!, input: UpdatePointOfInterestInput!,
    apiKey: String!): PointOfInterest!
  deletePointOfInterest(_id: ID!, apiKey: String!): String!

  generateApiKey(clientName: String!, password: String!): String!
}

```

Figure 7: Mutações Schema

GraphQL dispõe da possibilidade de criar queries e mutações e ainda criar diversos tipos para descrever todos os dados utilizados. Para o serviço foi criado um tipo ponto de interesse com vários campos adequados, inputs para serem utilizados noutras funções, dois queries (sendo o principal a procura de POIs) e quatro mutações.

Para começar é preciso denotar que para qualquer função da API ser utilizada é necessária a utilização de uma chave, chave essa que pode ser gerada utilizando a mutação 'generateApiKey' com a introdução do nome do cliente e uma passe. Esta chave pode ainda ser recuperada utilizando o query 'recoverApiKey'.

O query principal, 'searchPointsOfInterest', permite a utilização de três inputs para filtrar informação (mais um com o ID, para efeitos de desenvolvimento):

- localização: Representa um ponto 2D com latitude e longitude
- raio: Raio de procura a partir do ponto especificado
- nome da localização: Cidade, Região ou País

- categoria: Natureza, Comida, Cultura, Compras ou Marcos importantes

A seguir são apresentadas as validações feitas para os inputs.

- localização
 - latitude: entre -90º e 90º
 - longitude: entre -180 e 180
- raio: entre 100 e 50000 metros
- categoria: deve ser uma das especificadas em cima

Consoante os inputs fornecidos será disponibilizada a informação, i.e., os campos dos pontos de interesse encontrados. É importante notar que o ID, nome, localização e nome da localização são campos obrigatórios de um ponto de interesse que estão sempre disponíveis.

Resta referir as últimas três mutações, sendo que a primeira, já apresentada, corresponde ao 'generateApiKey'. A primeira destas mutações permite criar pontos de interesse com toda a informação dos campos do POI como input e retorna o ponto de interesse criado com uma mensagem de sucesso. Caso já exista um POI com o mesmo nome ou com uma localização muito próxima relativamente ao novo é retornada uma mensagem com indicação de POI já existente e também toda a informação desse POI.

Por último é ainda possível atualizar um POI inserindo o seu ID e a informação a ser atualizada ou remover um ponto de interesse igualmente pelo seu ID.

2.2.4 Resultados

Por forma a testar as diferentes funcionalidades da API foram realizados vários queries e mutações tanto a partir do IDE do Apollo, como a partir do terminal utilizando curls. É importante notar que todos os queries feitos em GraphQL são POSTs, onde toda a informação necessária é passada na data.

Os queries abaixo foram transcritos do IDE do Apollo já que este permite uma visualização mais legível da estrutura dos mesmos (e dos JSONs que são retornados por eles).

Os queries iniciais para utilização de chaves da API encontram-se abaixo.

```

1 mutation createApiKey {
2   generateApiKey(
3     clientName: "Tigas"
4     password: "1234"
5   )
6 }
7
8 query recoverApiKey {
9   recoverApiKey(
10    clientName: "Tigas"
11    password: "1234"
12  )
13 }
```

Foram realizados vários queries com informação de filtragem diferente para testar a função principal de pesquisa de POIs do endpoint da API. Abaixo encontra-se um exemplo que utiliza todos os filtros para encontrar informação exata de POIs.

```
1 query findPOIs {
2   searchPointsOfInterest(
3     apiKey: "Tigas:4712b0a1d771938c04e5cba078b0a889"
4     searchInput: {
5       location: {
6         type: "Point"
7         coordinates: [-8.653327, 40.630815]
8       }
9       radius: 10000
10      locationName: "Aveiro"
11      category: "Food"
12    }
13  ) {
14    _id
15    name
16    location {
17      coordinates
18    }
19    locationName
20    street
21    postcode
22    description
23    category
24    thumbnail
25  }
26 }
```

Sendo o que o ponto geográfico se situava em Aveiro, o raio de procura era de 10km, o próprio nome da localização era também Aveiro e a categoria era comida, o resultado final, em JSON, foi um único ponto de interesse, com as características exatas do que foi pedido:

```
1 {
2   "data": {
3     "searchPointsOfInterest": [
4       {
5         "_id": "5",
6         "name": "Restaurante Espeto do Sul",
7         "location": {
8           "coordinates": [
9             -8.649476,
10            40.635707
11          ]
12        },
13        "locationName": "Aveiro, Portugal",
14        "street": "R. de Sao Sebastiao 97",
15        "postcode": "3810-187",
16        "description": "O sabor unico do genuino rodizio brasileiro conquista Aveiro, sendo o mais tipico restaurante brasileiro onde cada refeicao e uma festa!",
17        "category": "Food",
18      }
19    ]
20  }
21 }
```

```

18     "thumbnail": "https://example.com/botanical-garden-
19     thumbnail.jpg"
20   }
21 }
22 }

```

Quanto a mutações, a criação também foi testada intensivamente. Depois de ser criado um novo ponto de interesse recebe-se, além de toda a informação do mesmo e o respetivo ID, a mensagem associada à sua criação (se foi criado com sucesso e, se não foi, o porquê).

```

1 mutation exCreation {
2   createPointOfInterest(
3     apiKey: "Tigas:4712b0a1d771938c04e5cba078b0a889"
4     input: {
5       name: "Forum Aveiro"
6       location: { type: "Point", coordinates: [-8.652582, 40.640957]
7     }
8     locationName: "Aveiro, Portugal"
9     description: "Otimo para fazer compras!",
10    thumbnail: "https://example.com/forumAveiro.jpg"
11    category: "Shopping"
12  }) {
13    poi {
14      _id
15      name
16      location {
17        coordinates
18      }
19      locationName
20      street
21      postcode
22      description
23      category
24      thumbnail
25    }
26    message
27  }
28 }

```

No query acima foi passada toda a informação para criar o novo POI, Forum Aveiro, com as característica obrigatórias (nome, local e nome de local) e mais algumas extra, como a categoria. Como resultado, a mensagem abaixo indica a criação correta do POI, e toda a informação relativa ao mesmo. Nota: Neste caso os campos street e postcode aparecem a null porque o criador não providenciou esta informação e os campos não foram guardados na base de dados.

```

1 {
2   "data": {
3     "createPointOfInterest": {
4       "poi": {
5         "_id": "665f31e74edb69b4b522867e",
6         "name": "Forum Aveiro",
7         "location": {
8           "coordinates": [
9             -8.652582,

```

```

10         40.640957
11     ],
12     },
13     "locationName": "Aveiro, Portugal",
14     "street": null,
15     "postcode": null,
16     "description": "Ótimo para fazer compras!",
17     "category": "Shopping",
18     "thumbnail": "https://example.com/forumAveiro.jpg"
19 },
20 "message": "Point of interest created successfully"
21 }
22 }
23 }

```

Se depois deste teste tentássemos criar um POI novamente com a mesma informação, como resultado obteríamos uma mensagem a indicar 'A point of interest with that name already exists, returning existing point of interest'. Se mudarmos o nome mas ficasse a mesma localização apareceria a mensagem 'A point of interest already exists within 100 meters, returning existing point of interest'. Em ambos os casos é retribuída a informação do POI já existente.

Finalmente, quanto à mutação update esta tem uma estrutura parecida às anteriores, com a diferença que é necessário especificar o ID do POI e, no input, os campos do mesmo que se pretendem atualizar. O delete pede apenas o ID do POI a apagar.

2.3 Serviço Metadados

2.3.1 Introdução

Este serviço permite gerir informação relacionada com eventos, desenvolvido na forma de uma API REST. Dada a necessidade de sincronização, este serviço comunica com os restantes:

- Serviço Calendarização (para efeitos de calendarização)
- Serviço Pontos de Interesse (para efeitos de localização)

Como a funcionalidade de armazenar ficheiros associados a eventos era desejada, foi criado um serviço extra:

- Serviço AmazonS3

2.3.2 Tecnologias

O serviço de Metadados utiliza Node.js para o servidor. Considerando que existe informação obrigatória e informação opcional, não havendo portanto formato fixo de tabelas, é utilizado MongoDB como base de dados. Este oferece maior flexibilidade e simplicidade através de coleções e documentos, e encontra-se bem integrado no ecossistema Node.js.

O serviço AmazonS3 utiliza o servidor Fake S3 recomendado pelo professor.

Este permite simular o funcionamento do AmazonS3 de maneira local e gratuita, através do sdk original `aws-sdk/client-s3`, para além de outros. Para este serviço não foi desenvolvido código, apenas os ficheiros `Dockerfile` e `deployment.yaml`

2.3.3 Esquema e Implementação

As funcionalidades oferecidas pelo serviço estão agrupadas conforme o recurso em que operam. Os endpoints disponibilizados retornam respostas adequadas aos pedidos realizados, com códigos de resposta e mensagens descritivas.

- **Chaves**

- **POST e1/keys** - Permite gerar uma chave API para autenticação de pedidos
- **PATCH e1/keys** - Permite revogar a chave API enviada no cabeçalho de autenticação do pedido, tornando esta inválida

- **Eventos**

- **POST e1/events** - Permite criar um novo evento, sendo este automaticamente adicionado ao calendário do criador. É possível ainda associar um ponto de interesse existente ou criar e depois associar um novo ponto de interesse.
- **GET e1/events** - Permite pesquisar eventos, de maneira geral ou através de parâmetros de pesquisa:
 - * **limit** - Para efeitos de paginação, tem valor default 25
 - * **offset** - Para efeitos de paginação, tem valor default 0
 - * **calendarid** - Todos os eventos contidos neste calendário
 - * **pointofinterestid** - Todos os eventos associados a este ponto de interesse
 - * **userid** - Todos os eventos criados por este utilizador
 - * **name** - Todos os eventos com este nome. Oferece correspondência parcial/total
 - * **organizer** - Todos os eventos com este organizador. Oferece correspondência parcial/total
 - * **category** - Todos os eventos com esta categoria. Oferece correspondência parcial/total
 - * **location** - Todos os eventos que decorrem nesta cidade/país. Oferece correspondência parcial/total
 - * **maxprice** - Todos os eventos até este preço (inclusive)
 - * **startdate** - Todos os eventos com esta data de início
 - * **beforedate** - Todos os eventos com data de fim até esta data (inclusive)
 - * **longitude** - Necessário para pesquisa por radius
 - * **latitude** - Necessário para pesquisa por radius

- * **radius** - Todos os eventos com localização até radius metros da localização dada por [longitude, latitude]. Pesquisa apenas considerada quando os três parâmetros longitude, latitude, radius são fornecidos
- **GET e1/events/{uuid}** - Permite pesquisar um evento específico
- **PUT e1/events/{uuid}** - Permite atualizar um evento específico
- **DELETE e1/events/{uuid}** - Permite eliminar um evento específico. Ao realizar esta ação, o evento é removido do calendário e os seus ficheiros associados são eliminados
- **PATCH e1/events/{uuid}/favorite** - Permite marcar/desmarcar um evento como favorito. Ao realizar esta ação, o evento é adicionado/removido do calendário do utilizador que realiza o pedido, e as estatísticas de favoritos são atualizadas

• Ficheiros

- **POST e1/files/{uuid}** - Permite armazenar um ficheiro associado a um evento
- **GET e1/files/{uuid}** - Permite obter um ficheiro associado a um evento
- **DELETE e1/files/{uuid}** - Permite eliminar um ficheiro associado a um evento

Em termos de implementação e funcionamento interno, o serviço apresenta os seguintes detalhes:

- Autenticação de pedidos através de chaves API enviadas num cabeçalho de autenticação. Estas têm o formato uuidv4, de modo a garantir a sua unicidade global. Para maior segurança, não são armazenadas diretamente na base de dados, passando primeiro por um processo de hashing:

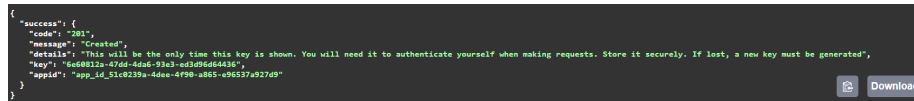
```
const key = uuidv4();
const hashedKey = crypto.createHash('sha256').update(key +
    config.server.secret).digest('hex');
```

- Autorização de ações que envolvam modificação de dados relativos a eventos (todos os PUT, DELETE, e POST de ficheiros). Feito através da identificação do utilizador que realiza o pedido, tendo de ser este o criador original
- Implementação de um mecanismo de timeout/retry em chamadas a serviços externos. Este procura evitar conexões penduradas através da especificação de um timeout e a possibilidade de novas tentativas caso ocorram problemas de conexão. Existe um limite de tentativas e o timeout é dinâmico, aumentando a cada nova tentativa

- Logs persistentes de todos os erros e pedidos/respostas processados. Todos os dias onde exista pelo menos um pedido feito ao servidor, é criado um novo ficheiro para erros e um novo ficheiro para pedidos/respostas, identificados pelas datas, com os restantes a serem zipados, reduzindo o espaço de armazenamento ocupado. De modo a facilitar a análise, cada par pedido-erro e pedido-resposta é identificado por um uuidv4
- Validação do tipo/formato/valor dos dados recebidos em pedidos, antes do seu processamento, através de middleware.
- Paginação para evitar problemas de serviço relacionado com grandes quantidades de dados
- Documentação desenvolvida em Swagger

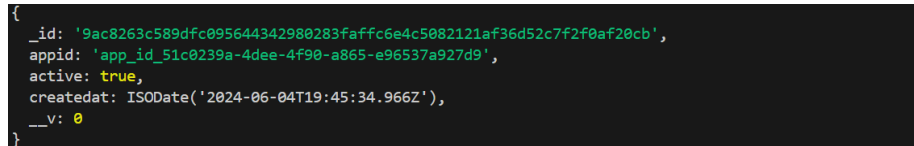
2.3.4 Resultados

Em relação a chaves API, é possível verificar o resultado na coleção *apikeys* da base de dados. A chave é fornecida na sua forma original apenas uma vez na resposta ao pedido, sendo armazenada na sua forma hashed (no campo *_id*).



```
{
  "success": {
    "code": "200",
    "message": "Created",
    "details": "This will be the only time this key is shown. You will need it to authenticate yourself when making requests. Store it securely. If lost, a new key must be generated",
    "key": "66d8812e-47de-4de6-93e3-ea3296d6445d",
    "appid": "app_id_51c0239a-4dee-4f90-a865-e96537a927d9"
  }
}
```

Figure 8: Chave API gerada



```
{
  _id: '9ac8263c589dfc095644342980283faffc6e4c5082121af36d52c7f2f0af20cb',
  appid: 'app_id_51c0239a-4dee-4f90-a865-e96537a927d9',
  active: true,
  createdat: ISODate('2024-06-04T19:45:34.966Z'),
  __v: 0
}
```

Figure 9: Chave API armazenada

Em relação a metadados de eventos, é possível verificar o resultado na coleção *events* da base de dados. O campo *userid* refere o utilizador que criou o evento. O campo *calendarid* refere o calendário desse utilizador com os dados de calendarização associados ao evento. O campo *pointofinterestid* refere o ponto de interesse com os dados de localização associados ao evento.

```
{
  _id: 'cac80a76-72a5-4845-a630-10fabe049caa',
  userid: '1',
  organizer: 'Portuguese Football Federation',
  category: 'sports',
  contact: 'fpf@support.com',
  price: 60,
  currency: 'EUR',
  favorites: 6,
  maxparticipants: 50095,
  currentparticipants: 50000,
  calendarid: '1',
  pointofinterestid: '5f8f4b3b9b3e6b1f3c1e4b1a'
},
```

Figure 10: Dados de eventos

Em relação a estatísticas de favoritos, é possível verificar o resultado na coleção *favorites* da base de dados. Cada utilizador pode ter apenas uma entrada por cada evento, de modo a prevenir entradas repetidas. Esta entrada é criada apenas uma vez, quando o utilizador marca um evento como favorito pela primeira vez, sendo o status posteriormente atualizado conforme necessário.

```
{
  _id: ObjectId('664dcf88a5f70d52f62202d8'),
  eventid: 'cac80a76-72a5-4845-a630-10fabe049caa',
  userid: '1',
  favoritestatus: true
},
{
  _id: ObjectId('664dcf88a5f70d52f62202d9'),
  eventid: '5034bfc8-e761-4426-9fa4-c1a276bf163f',
  userid: '1',
  favoritestatus: false
},
{
  _id: ObjectId('664dcf88a5f70d52f62202da'),
  eventid: '4c6ce530-c1f2-424f-a261-2fabaf533679',
  userid: '1',
  favoritestatus: true
},
```

Figure 11: Estatísticas de favoritos

Em relação a logs, é possível verificar o resultado no diretório *app/event-logs* do serviço Metadados. Existem logs separados por erro e por informação, com os logs de dias anteriores zipados para poupar espaço de armazenamento. Para cada par de mensagens, o campo *messageID* é igual, permitindo uma análise sequencial de eventos ocorridos.

```

root@event-service-544b68c849-7tlbn:/app/event-logs# ls
error  info
root@event-service-544b68c849-7tlbn:/app/event-logs# cd info/
root@event-service-544b68c849-7tlbn:/app/event-logs/info# ls
info_2024-05-28.log.gz  info_2024-05-29.log.gz  info_2024-06-02.log.gz  info_2024-06-04.log
root@event-service-544b68c849-7tlbn:/app/event-logs/info# cat info_2024-06-04.log
{
  messageID: '2718dc0f-d39e-4021-abb7-aa10e838e8e3',
  messageType: 'REQUEST',
  message: 'GET /e1/events?limit=25&offset=0',
  fromIP: '10.42.0.221',
  content: undefined,
  params: {},
  query: { limit: '25', offset: '0' },
  body: {},
  level: 'info',
  service: 'Event-service',
  timestamp: '2024-06-04 19:16:16'
}
{
  messageID: '2718dc0f-d39e-4021-abb7-aa10e838e8e3',
  messageType: 'RESPONSE',
  message: 'GET /e1/events?limit=25&offset=0',
  toIP: '10.42.0.221',
  status_code: 401,
  level: 'info',
  service: 'Event-service',
  timestamp: '2024-06-04 19:16:16'
}
root@event-service-544b68c849-7tlbn:/app/event-logs/info# █

```

Figure 12: Log de eventos

Em relação a arquivos associados a eventos, é possível verificar o resultado no diretório `app/fake-s3/Event_Service` do serviço AmazonS3. Cada evento pode ter apenas um arquivo associado, com tamanho máximo de 10MB e do tipo `/jpeg`.

```

/app/fake-s3/Event_Service # ls
event_21cd4b9a-8ef0-4824-9038-88eb038ae16d.jpeg  event_4c6ce530-c1f2-424f-a261-2fabaf533679.jpeg  event_8a53079e-8093-4022-8e76-10543e42c03e.jpeg
event_41378065-359b-42f3-84e5-8b29dc20c532.jpeg  event_5034bfc8-e761-4426-9fa4-c1a276bf163f.jpeg  event_cac80a76-72a5-4845-a630-10fab049caa.jpeg
/app/fake-s3/Event_Service # █

```

Figure 13: Arquivos no sistema de armazenamento

Code	Details
200	Response body Download file Response headers <pre> access-control-allow-origin: * content-disposition: attachment; filename=event_cac80a76-72a5-4845-a630-10fab049caa.jpeg content-length: 65668 content-type: image/jpeg date: Tue, 04 Jun 2024 20:47:58 GMT etag: W/"6e984-d51foug8s+dc5kh0eWVTVShhZa" server: nginx/1.25.1 x-powered-by: Express </pre>

Figure 14: Obtenção de um arquivo do sistema de armazenamento

Em relação a documentação, é possível verificar o resultado em: <http://grupo4-egs-deti.ua.pt/e1/docs>
Pode ser usada a chave 93489d58-e2cf-4e11-b3ac-74381fee38ac para efeitos de autenticação.

		Authorize 
Keys Operations available for Keys ^		
POST	/e1/keys	Generate a key
PATCH	/e1/keys	Revoke a key 
Events Operations available for Events ^		
POST	/e1/events	Create a new event 
GET	/e1/events	Get all events 
GET	/e1/events/{uuid}	Get an event 
PUT	/e1/events/{uuid}	Update an event 
DELETE	/e1/events/{uuid}	Delete an event 
PATCH	/e1/events/{uuid}/favorite	Add/Remove a favorite from an event 
Files Operations available for Files ^		
POST	/e1/files/{uuid}	Upload a file 
GET	/e1/files/{uuid}	Get a file 
DELETE	/e1/files/{uuid}	Delete a file 

Figure 15: Documentação do serviço

3 Kubernetes

A aplicação encontra-se deployed na rede kubernettes disponibilizada, usando o DNS *grupo4-egs-deti.ua.pt*

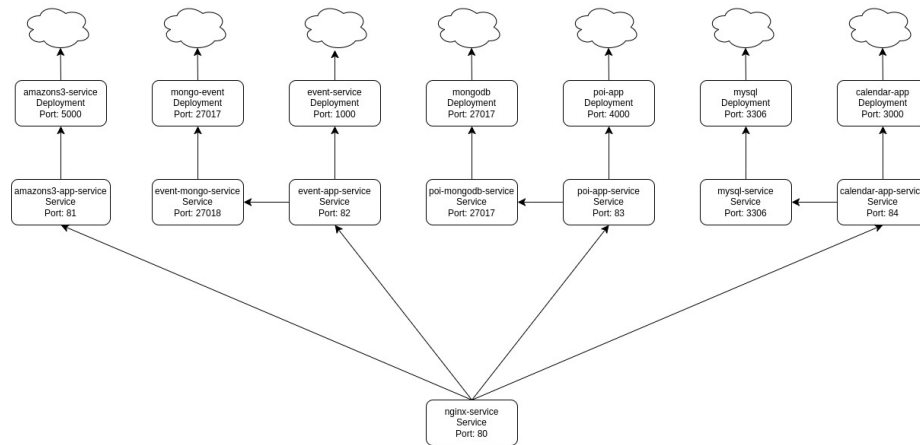


Figure 16: Arquitetura de deployment

O ficheiro deployment encontra-se disponível: <https://github.com/EGS-Tourist-Guide/CalendarAPI/blob/main/deployment.yaml>

☐ Name	Namespace	Type	Cluster IP	Ports	External IP	Select	Age	Status
☐ event-mongo-service	egs-touristguide	ClusterIP	10.43.107.118	27018:27017/TCP	-	app=	6d10h	Active
☐ mysql-service	egs-touristguide	ClusterIP	None	3306/TCP	-	app=	6d10h	Active
☐ nginx-service	egs-touristguide	ClusterIP	10.43.113.213	80/TCP	-	app=	6d10h	Active
☐ poi-mongodb-service	egs-touristguide	ClusterIP	None	27017/TCP	-	app=	6d10h	Active
☐ amazonS3-app-service	egs-touristguide	LoadBalancer	10.43.154.26	81:32727/TCP	-	app=a	6d10h	Pending
☐ calendar-app-service	egs-touristguide	LoadBalancer	10.43.220.189	84:32347/TCP	-	app=c	6d10h	Pending
☐ event-app-service	egs-touristguide	LoadBalancer	10.43.225.96	82:31163/TCP	-	app=e	6d10h	Pending
☐ poi-app-service	egs-touristguide	LoadBalancer	10.43.179.129	83:31555/TCP	-	app=p	6d10h	Pending

Figure 17: Pods em execução

4 Demo

4.0.1 Video Demo

O video demonstrativo pode ser visualizado no seguinte link:

- <https://youtu.be/uYetrFXB0Sw?si=wnclssMrN9VrBJpm>

4.0.2 Implementação

Nesta demonstração vamos interligar todos os serviços de forma a conseguir interagir por meio de uma interface onde o utilizador pode organizar as suas viagens e com o propósito de fazer isto em "real time". Para esse efeito decidimos criar uma aplicação nativa em android studio.

Esta aplicação é dividida essencialmente em 7 partições:

1. Login, onde conforme o mail e a password inseridos na autenticação faz um request pelo endpoint *"http://grupo4-egs-deti.ua.pt/login"* enviando estes dois parâmetros preenchidos com o propósito de receber a sua identificação caso já exista, criar uma nova caso ainda não exista ou de confirmar se dão match e se é válida.
2. Lista de pontos de interesse, esta página vai chamar o endpoint *"http://grupo4-egs-deti.ua.pt/graphql"* onde vem toda a informação dos pontos de interesse possíveis.
3. Aplicação de filtros para facilitar a pesquisa em termos de localização, categorias e de coordenadas que estão dependentes do raio, para este último método disponibilizamos um auxílio de um mapa onde é possível selecionar um ponto no mapa e a partir daí recolher as coordenadas depois em conjunto com o raio, enviar com estas opções no endpoint.
4. Lista de eventos para cada ponto de interesse, quando selecionado um ponto de interesse é redirecionado para a página de lista de eventos associado ao mesmo. Entre páginas é guardado o identificador do utilizador, do calendário e do ponto de interesse selecionado.
5. Apresentação de detalhes de um evento. Temos a possibilidade de guardar um determinado evento que o utilizador esteja a ler e que goste com recurso a um PATCH, este vai atualizar o estado "isFavorite" do evento passando como boolean a true no endpoint *"http://" + serverIp + "/e1/events/" + eventId + "/favorite"* e o mesmo vai ser adicionado ao calendário.
6. Calendário com as marcações dos eventos que foram guardados. Tem como propósito relembrar e confirmar todas as suas marcações. Para trabalho futuro poderia ser adicionado um serviço de notificação quando a data se aproxima. Para este endpoint foi usado um GET dando o identificador do calendário para mostrar os eventos associados ao calendário em específico

"http://grupo4-egs-deti.ua.pt/e1/events?limit=25offset=0calendarid=" + calendarIdReq"

7. Página com a lista de eventos para um dia em específico, temos a possibilidade de remover um evento guardado no calendário e faz-se tal como na página de detalhes do evento que fazia um PATCH para atualizar só que desta vez passamos o "isFavorite" a false.

4.1 Endpoint Login

Nas duas imagens de baixo 18 19 apresentamos duas formas de entrar na aplicação, uma com uma conta já existente "marta@example.com" e outra criada no momento em que dá "Login" por não ter aquele registo na base de dados.

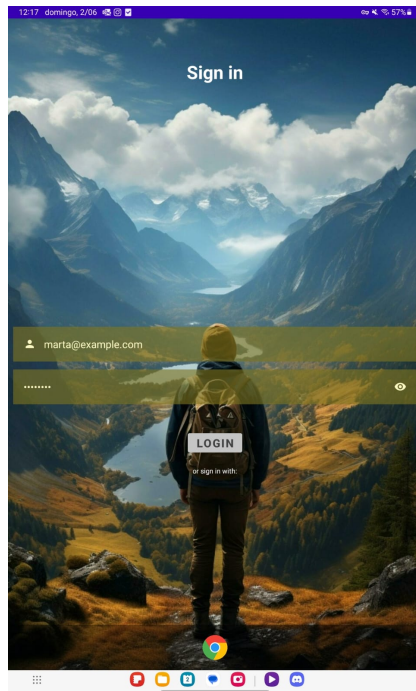


Figure 18: Login existent

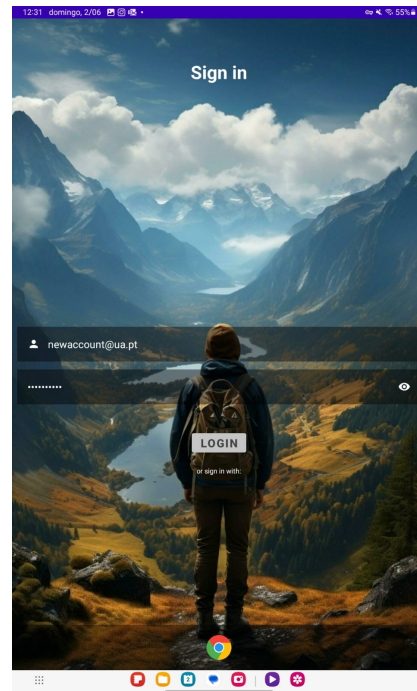
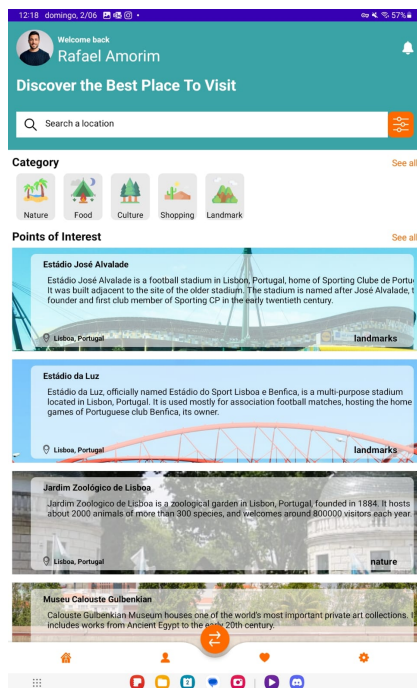


Figure 19: Nova conta

Figure 20: Login

4.2 Endpoints Pontos de Interesse

Na primeira imagem 21a estão presentes todos os pontos de interesse disponíveis não sendo preciso aplicados filtros ao entrar nesta página. Já nas imagens seguintes apresentamos diversas formas de aplicar os filtros e dessa forma a lista de itens vão variando consoante as opções selecionadas.

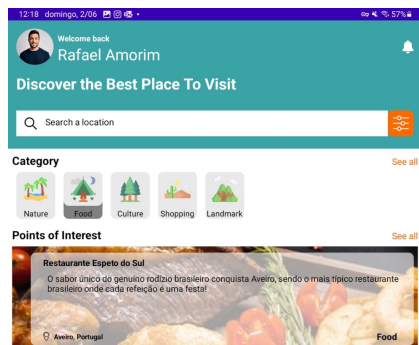


(a) Todos os pontos de interesse

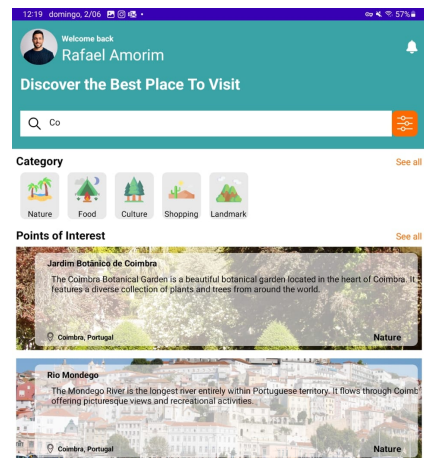


(b) Filtro da categoria de landmarks

Figure 21: Todos os pontos de interesse e categoria landmarks selecionada



(a) Filtro da categoria de comida

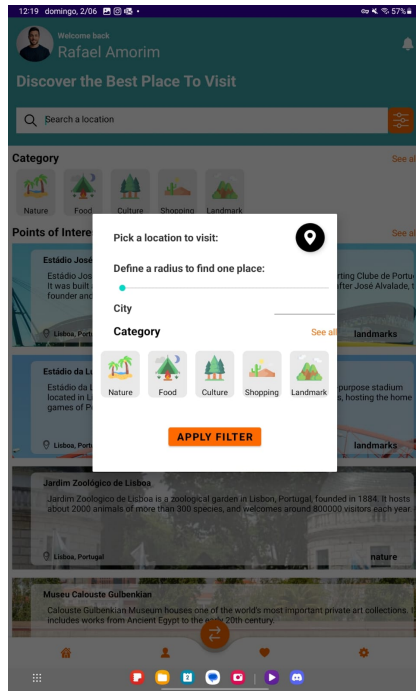


(b) Filtro no lupa de pesquisa (on text changed)

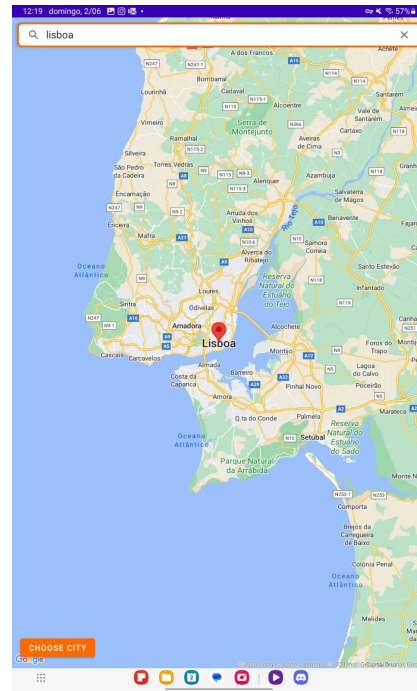
Figure 22: Filters

Na imagem de cima 22a temos seleccionadas as opções de comida que foi seleccionada aparecendo a negrito a sua seleção, já na imagem da direita 22b faz procuras dos pontos de interesse à medida que vai escrevendo na barra de pesquisa tanto localizações como categorias.

Outra forma de procurar por pontos de interesse é através de coordenadas, as quais dependem de um raio para recolher todos os locais na base de dados que pertençam a essa área. Para obter estas coordenadas como podemos ver nas imagens de baixo basta clicar no icon preto com um "pin" 23a, sendo o utilizador redirecionado para um mapa 23b onde deve escrever o sítio que deseja visitar e depois faz "choose city" para guardar as coordenadas desse local.

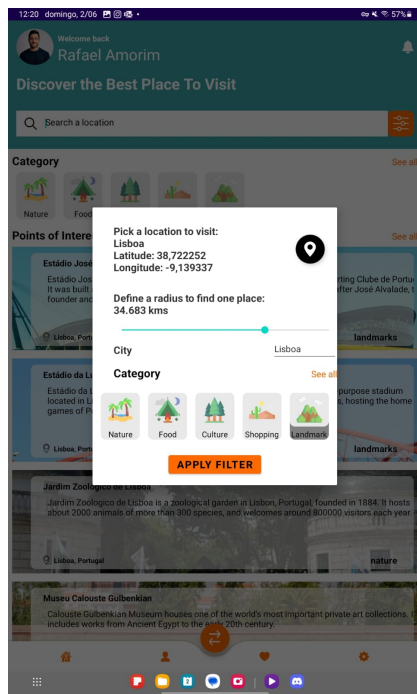


(a) Popup sem filtros

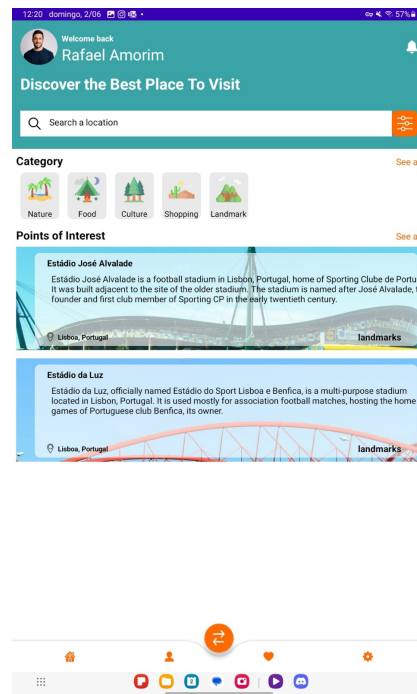


(b) Mapa para obter coordenadas

Figure 23: Popup vazio



(a) Popup com filtros



(b) Depois de aplicar o filtro do popup

Figure 24: Popup preenchido

Ao clicar no botão "APPLY FILTER" é apresentado os resultados como se pode ver na imagem da direita 24b.

4.3 Endpoints Eventos

Quando escolhido o ponto de interesse, este com o seu id associado é passado para a próxima página e procura todos os eventos nesse local como podemos ver na imagem de baixo à esquerda 25a. Já a da direita 25b serve para apresentar toda a informação detalhada relacionada com o evento escolhido tal como, o preço, dia e hora, descrição, entre outros. Aqui também é possível clicar no botão "Add Event to Calendar" para guardar este evento no calendário.

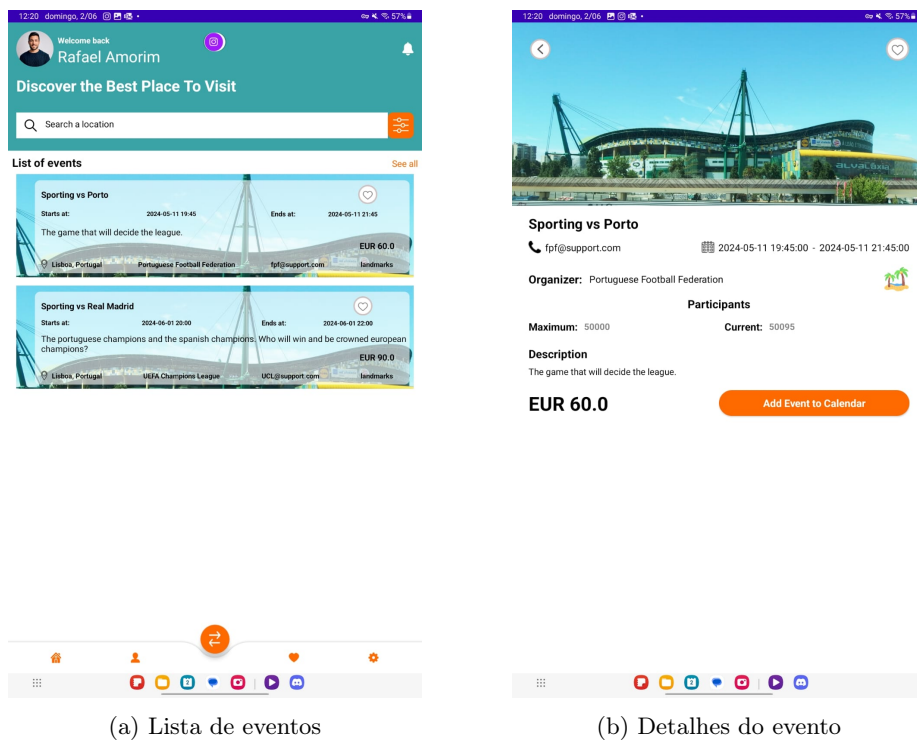
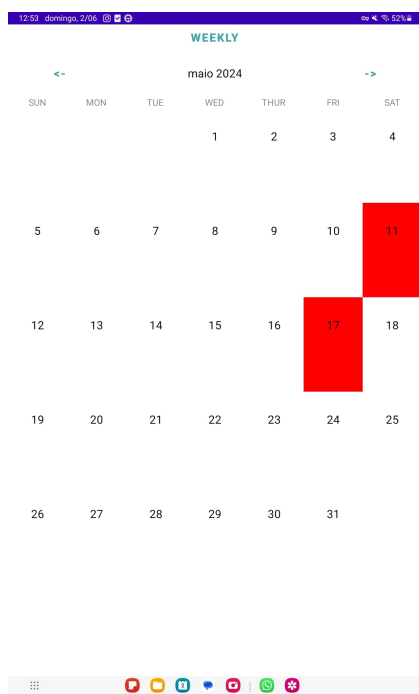
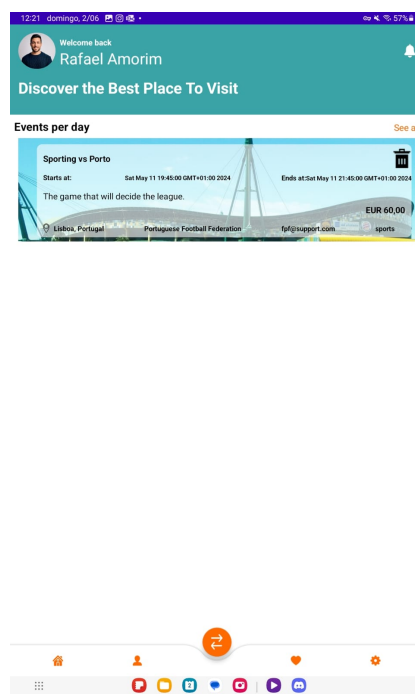


Figure 25: Chamada da lista de eventos e adição ao calendário

Na página dos detalhes tínhamos o dia 11 de maio para ser marcado e como podemos ver agora no calendário ?? aparece o dia 11 a vermelho e ao clicar neste mostra a mesma informação resumida para relembrar o utilizador de qual o evento que tem para aquele dia.



(a) Adição do evento ao calendário



(b) Event no dia 11 de maio

Figure 26: Chamada da lista de eventos e adição ao calendário

Por fim, ao selecionar o dia 17 de maio como temos em cima 27 apresentado a vermelho, inicialmente temos dois eventos nesse dia. Clicando no caixote do lixo, é executada uma atualização na base de dados pelo endpoint com um PATCH através do seguinte link `"http://" + serverIp + "/e1/events/" + eventId + "/favorite"`, onde é enviado o estado de "isFavorite" a false, removendo da lista dos guardados/favoritos. Voltando a fazer o mesmo com o segundo item, conseguimos observar na imagem 29 que a marcação a vermelho desaparece do calendário.

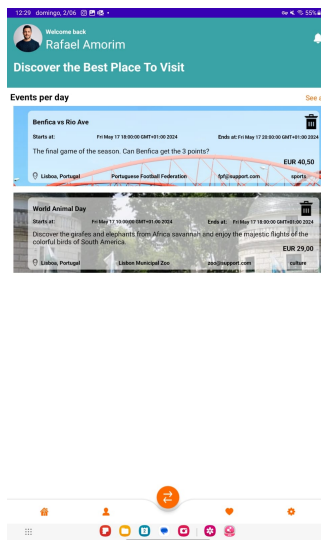


Figure 27: Eventos por dia

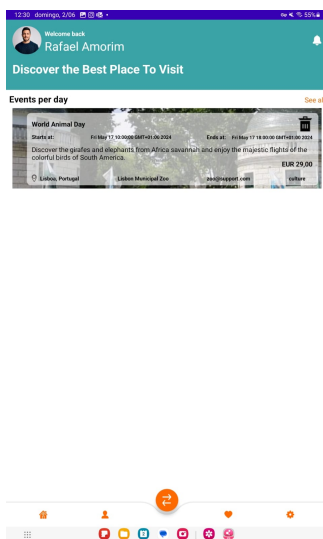


Figure 28: Evento do Benfica saiu dos favoritos

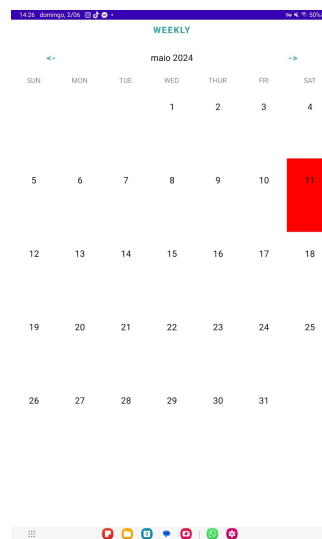


Figure 29: Sem eventos no dia 17

Figure 30: Resultado dos eventos eliminados

5 Autoavaliação

Dado que os requisitos do nosso projeto foram alcançados de acordo com os critérios definidos inicialmente, a autoavaliação do grupo é 17.

Acreditamos que a divisão de tarefas foi feita de forma equitativa, com cada membro contribuindo de maneira significativa e equitativa para o projeto.